

# LINQ Cast Method in C# with Examples:

In this article, I will discuss the **LINQ Cast Method in C#** with Examples. Please read our previous article discussing the [LINQ ToDictionary Method in C#](#) with Examples. Like the LINQ ToList, ToArray, and ToDictionary, the Cast Method also belongs to the conversion operators category.

## LINQ Cast Method in C#:

The LINQ Cast method in C# is used to cast elements of a non-generic collection to a specified type. It's a part of the System.Linq namespace can be applied to any type that implements IEnumerable, converting it into an IEnumerable<T> where T is the target type. The Cast method is particularly useful when working with not strongly typed collections, allowing you to apply LINQ queries that require a specific type.

The LINQ Cast Method in C# is used to cast all the elements of a collection (**System.Collections.IEnumerable**) to a specified type and then return a new System.Collections.Generic.IEnumerable<T> collection containing all the source sequence elements cast to the specified type. This method uses deferred execution. The signature of this method is given below.

```
public static IEnumerable<TResult> Cast<TResult>(this IEnumerable source);
```

### Parameters:

1. **source:** The **System.Collections.IEnumerable** that contains the elements to be cast to type **TResult**.

### Type Parameters:

1. **TResult:** The type to cast the elements of the source.

### Exceptions:

1. It will throw the **System.ArgumentNullException** when the source is null.
2. It will throw the **System.InvalidCastException** when an element in the source sequence cannot be cast to the specified type TResult.

### Returns:

1. It returns a System.Collections.Generic.IEnumerable contains each element of the source sequence cast to the specified type.

## Example to Understand LINQ Cast Method in C#:

In the following example, the source sequence contains 3 elements, all of which are type integers. So, all these elements are cast into integers. So, while casting using the Cast method, we need to specify the type we want to cast.

```

using System.Linq;
using System.Collections.Generic;
using System;
using System.Collections;
namespace ConversionOperators
{
    class Program
    {
        public static void Main()
        {
            ArrayList list = new ArrayList
            {
                10,
                20,
                30
            };
            IEnumerable<int> result = list.Cast<int>();
            foreach (int i in result)
            {
                Console.WriteLine(i);
            }
            Console.ReadKey();
        }
    }
}

```

Output: 10 20 30

## LINQ Cast Method Key Points

- **Type Casting:** Cast attempts to cast every element in the collection to the specified type. If any element cannot be cast to the specified type, an `InvalidCastException` is thrown at runtime.
- **Use with Non-Generic Collections:** Cast is mainly used with non-generic collections like `ArrayList` or non-generic `IEnumerable`. It's unnecessary for generic collections like `List<T>`, as they are already strongly typed.
- **Deferred Execution:** The casting operation is performed when you iterate over the resulting `IEnumerable<T>`, not when the `Cast` method is called.
- **Combining with LINQ Queries:** After casting, you can apply standard LINQ query operators (like `Where`, `Select`, `OrderBy`) on the collection.

## Example2:

In the following example, we have created a non-generic (`ArrayList`) collection that contains integer and string values. Then, we cast that collection to `IEnumerable<int>` using the `Cast<int>` method. At runtime, while casting the values, it will throw **`System.InvalidCastException`** runtime exception when it tries to cast the string value.

```

using System.Linq;
using System.Collections.Generic;
using System;

```

```

using System.Collections;
namespace ConversionOperators
{
    class Program
    {
        public static void Main()
        {
            ArrayList list = new ArrayList
            {
                10,
                20,
                30,
            };
            //The following statement throws System.InvalidCastException
            list.Add("40");
            IEnumerable<int> result = list.Cast<int>();
            foreach (int i in result)
            {
                Console.WriteLine(i);
            }
            Console.ReadKey();
        }
    }
}

```

### Example3:

In the following example, the source sequence is null. So, when we run the application, it will throw the **System.ArgumentNullException**.

```

using System.Linq;
using System.Collections.Generic;
using System;
using System.Collections;
namespace ConversionOperators
{
    class Program
    {
        public static void Main()
        {
            ArrayList list = null;
            //Throws System.ArgumentNullException
            IEnumerable<int> result = list.Cast<int>();
            foreach (int i in result)
            {
                Console.WriteLine(i);
            }
            Console.ReadKey();
        }
    }
}

```

}

## When to Use the LINQ Cast Method in C#?

The LINQ Cast method in C# is particularly useful in scenarios where you have a non-generic collection and need to perform operations that require the elements to be of a specific type. Here are some typical use cases:

- **Working with Non-Generic Collections:** When dealing with older non-generic collections like ArrayList or Hashtable, where elements are stored as objects, Cast can be used to convert these elements into a specific type for further processing.
- **Applying LINQ Queries to Non-Generic Collections:** To use LINQ query operators (like Where, Select, OrderBy) on non-generic collections, you must first cast the elements to a specific type. Cast enables this transition.
- **Data Transformation:** In scenarios where data retrieved from sources like databases, file systems, or external services comes in a non-generic format, Cast can be used to transform this data into a strongly typed format for ease of manipulation.
- **Interoperating with Older .NET Code:** If you're working with code predating generics in .NET and need to integrate it with newer, strongly typed code, Cast can serve as a bridge by converting collections to the required generic types.
- **Type-Safe Enumeration:** When you need to enumerate through a non-generic collection while performing type-specific operations on each element, Cast ensures that the elements are of the expected type, thus enabling type-safe operations.
- **Combining Collections of Different Types:** If you have multiple non-generic collections that you need to combine or process uniformly, Cast can be used to ensure all collections are of the same type before combining or querying them.

It's important to note that Cast will throw an `InvalidCastException` at runtime if any element in the collection cannot be cast to the specified type. Therefore, its use assumes that you know the types of elements in the collection and are confident that all elements can be safely cast to the target type.

In the next article, I will discuss the [Differences Between Cast and OfType Methods in C#](#) with Examples. In this article, I explain the need and use of the **LINQ Cast Method in C#** with Examples. I hope you understand the need and use of the Cast Method in C#.